The Name of the Title Is Hope

Anonymous Author(s)

Abstract

LLM agents with web access are increasingly becoming the default for user-facing question answering. Reliable evaluation in this setting requires tests that remain unseen at train time and cannot be answered via verbatim lookup at inference. However, standard practice still relies on static benchmarks that age quickly and are vulnerable to contamination. We analyze two channels: pretraining leakage, where test items appear in the pretraining data, and retrieval-time leakage, where the system surfaces the exact target answer verbatim during evaluation and reports it rather than demonstrating genuine problem solving. We uncover evidence across five QA datasets for both channels and show that static evaluation can overestimate capability. We introduce DynamicWebQA, a dynamic evaluation framework for web-RAG that constructs questions and verifiable answers at evaluation time from current web sources. The framework enforces multi-document grounding, records evidence chains, and offers controllable complexity via source-set size, number of hops, and reasoning constraints. By generating fresh test instances, DynamicWebQA reduces memorization risk, mitigates retrieval-time leakage, and naturally covers time-sensitive queries.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:

Anonymous Author(s). 2018. The Name of the Title Is Hope. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, **??** pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Benchmark leakage refers to the contamination that occurs when a model has prior exposure to a benchmark's test items during pretraining, leading to inflated evaluation scores and undermining the benchmark's validity [**? ?**]. Since most LLMs are pretrained indiscriminately on publicly available web content, there is a high likelihood that they might have encountered benchmark datasets hosted on platforms like HuggingFace, GitHub, Kaggle, and similar sites. If the base LLM used in a WebQA agent has already seen the test examples during pretraining, the evaluation effectively becomes a case of testing on the training set — severely compromising the credibility of the reported results.

Dynamic benchmarking is an emerging evaluation paradigm that generates or adapts test samples at evaluation time, so models are scored on fresh, unseen data and testset contamination is avoided [**? ? ? ? ? ?**]. That means the agent does not see the same test sample twice, effectively addressing issues such as LLM memorization.

In this work, we refer to information that can change over time, such as the stock price of Tesla, as evolving information. Arguably, this type of content could be labeled as dynamic content or temporal content, but that might cause confusion, since we are already using the term "dynamic" to describe an evaluation paradigm (dynamic benchmarking), and "temporal" might lead readers to associate it with time-series data. To avoid such confusion, we adopt the term evolving information to specifically denote factual content that may shift over time without implying a particular format or evaluation framework.

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

2 Related Work

Open-Domain QA (ODQA). The ODQA task studies how to find and use evidence from large corpora to answer free-form questions. A sequence of datasets marked key shifts in task design: SQuAD [**?**] established span-based reading comprehension; Natural Questions [**?**] moved to real search queries; TriviaQA [**?**] stressed compositional, trivia-style questions with substantial lexical and syntactic variation; and HotpotQA [**?**] formalized multi-hop reasoning across multiple documents with sentence-level supporting facts and comparison questions, while including distractors to test focus under noise. Models such as fusion-in-decoder (FiD) [**?**] showed that aggregating many retrieved passages is a powerful way to synthesize scattered evidence.

Another important dimension in ODQA is *time*. RealTime QA [**?**] and FreshQA [**?**] evaluate whether systems can answer what is true *right now*, revealing failure modes when indices are stale or retrieved content is outdated or contradictory. These settings highlight needs that go beyond span extraction: recognizing when the answer has changed, when a question carries a false premise, and when recent evidence is required over older but higher-ranked documents. Notably, surface simplicity does not imply task simplicity. SimpleQA [**?**] demonstrates that short, seemingly straightforward questions can still require precise world knowledge or multi-step reasoning, indicating that difficulty is not well represented by length or phrasing.

Early ODQA followed a retrieve-then-read paradigm with extractive readers over IR-retrieved text. DrQA used TF-IDF/BM25 over Wikipedia coupled with a neural span extractor [**?**]. BERTserini then paired BM25 with a BERT reader to push end-to-end accuracy [**?**]. To support multi-hop questions, subsequent work leveraged structured signals. **?**] learned to retrieve reasoning paths over the Wikipedia hyperlink graph to connect supporting pages. **?**] introduced document–entity heterogeneous graphs (e.g., GraftNet), which aggregated evidence across passages and entities for open-domain QA. Dense retrieval reduced lexical mismatch: ORQA learned a latent retriever from QA pairs [**?**], and DPR's dual-encoder with supervised negatives yielded large gains over strong BM25 baselines [**?**]. These systems still relied on static indices, motivating later generation-conditioned approaches like RAG.

Retrieval-Augmented Generation (RAG). Modern RAG approaches follow a common pipeline – given a query, retrieve top-k passages from a non-parametric memory and condition a generator on them. Foundational formulations include RAG [**?**], REALM [**?**], RETRO [**?**], and ATLAS [**?**], which together demonstrate that query-time retrieval improves knowledge-intensive tasks while enabling the external memory to be updated without re-training the parametric model. Recent surveys study components and variants of this paradigm [**?**].

In the literature, RAG is distinguished from broader tool-augmented generation – RAG requires *query-conditioned retrieval* whose contents directly condition the generator. Calculators, code execution, or static always-prepended context are typically treated as non-retrieval tools [**?**]. In practice, the retrieval index can be the open web or enterprise/local corpora, and the mechanism can be lexical (e.g., BM25) or dense; the generator then aggregates evidence across multiple passages [**? ?**].

Beyond single-shot RAG, agentic frameworks let models iteratively issue searches, follow links, and cite sources. WebGPT augments an LLM with browsing and requires collecting references during answering [**?**]. ReAct further couples reasoning traces with tool calls (e.g., search, calculator) in a unified prompting loop, enabling multi-hop planning and verification [**?**]. These approaches often embed RAG steps inside a broader plan-act-observe loop, allowing complex queries that require cross-document synthesis.

Data Contamination. Data contamination arises when evaluation items (or close paraphrases and answer-bearing spans) leak into the corpora used for pretraining, instruction tuning, or fine-tuning. In such cases, test behaviour can devolve into *lookup* of memorized strings rather than the intended inference or problem solving, inflating reported performance and obscuring brittleness [**? ?**]. Prior studies connect leakage to memorization dynamics: larger models memorize more and earlier in training; duplication and longer prompts amplify verbatim regurgitation; and model capacity allows substantial unintended storage of datapoint-specific information [**? ? ?**]. At the same time, verbatim completion is not definitive evidence of membership – models can perfectly complete sequences never explicitly seen during training – implying membership tests must be interpreted cautiously [**?**]. The measurable impact of contamination is further mediated by dataset construction, model scale, and properties of pretraining data [**?**].

?] organize detection techniques by *data availability*. With candidate corpora available (*open-data*), methods compare evaluation items against sources via instance-/dataset-level span matching, embedding similarity, or paraphrase detection. When training data are unavailable (*closed-data*), researchers rely on indirect evidence such as performance analyses, membership-inference attacks, and instance-level probes of memorization and model confidence. Despite blind spots, approaches based on span matching remain the practical default in most foundational efforts [**? ? ? ? ? ?**] – as they are auditable and more importantly, tractable at web scale. Auditing a single question or sentence against a large crawl is a needle-in-a-haystack search; doing so for entire benchmarks across billions of documents renders more sophisticated but compute-heavy detectors infeasible. Moreover, behavioural signals alone rarely certify contamination [**? ?**].

Retrieval-augmented and agentic QA introduce a new concern – these systems may retrieve and quote target items from web or enterprise indices at *run time*, creating search-time data contamination that boosts scores without demonstrating reasoning [**?**]. **?**] show that even small leakage rates (e.g., on the order of a few percent) can reorder competitive leaderboards once online access is enabled, undermining trust relative to purely offline evaluation.

Dynamic Benchmarking. Most benchmarks for agentic or retrieval-augmented QA remain static and publicly accessible (e.g., HotpotQA), which makes them susceptible to contamination and memorization [**? ? ? ? ?**]. As models and training corpora grow, leakage can inflate reported scores on static benchmarks. This has motivated *dynamic* evaluation paradigms that are more robust towards contamination and memorization [**? ? ? ?**].

Early dynamic platforms such as Dynabench and Dynaboard introduced human- and model-in-the-loop data collection [**? ? ?**], but their reliance on crowdsourcing limits scalability. More recent

approaches leverage domain-specific settings (e.g., AndroidWorld for interactive agents [?]) or graph-based generation to create evaluation samples with controllable complexity and cross-round consistency [? ? ?]. While graph-anchored methods have proven effective in tasks like reasoning and KGQA [? ?], their application to ODQA is still limited, a gap we address in this work.

3 Modeling Contamination in RAG-Based QA

RAG systems combine a parametric model M with external sources retrieved at inference. This creates two leakage channels: (i) *pre-training contamination*, where an evaluation item (or a close paraphrase / answer-bearing span) appears in the corpus used to pre-train M; and (ii) *retrieval-time contamination*, where the retriever surfaces such material during evaluation, enabling lookup rather than reasoning. Our experiments instantiate web-enabled RAG "Web-RAG", but the formalism below applies to RAG broadly.

3.1 Definitions

Let M be pretrained on corpus D_M. Let D_{eval} contain items $x = (q, a)$ with question q and answer a. Let $\mathcal{R}(q)$ be the set of documents returned by the retriever for query q. Following ?], let f be a similarity function and τ a threshold; τ=1 corresponds to *strict/verbatim* contamination.

We study both *question-only* and *question+answer* leakage, noting that even input-only (question-only) leakage can shift measured performance [?]. We consider two views of an item:

$$\pi_{\text{in}}(q, a) = q \qquad (\textit{question-only}, \text{a.k.a. input-only}),$$

$$\pi_{\text{in+lab}}(q, a) = q \,\|\, a \qquad (\textit{question+answer}, \text{a.k.a. input+label}).$$

Contamination criterion. For a channel $C \in \{\text{pre}, \text{ret}\}$ and view $v \in \{\text{in}, \text{in+lab}\}$, we say D_{eval} is contaminated if

$$\exists\, x \in D_{\text{eval}},\ \exists\, d \in \mathcal{S}_C(x) \ \text{ s.t. } \ f(\pi_v(x), d) \geq \tau, \tag{1}$$

with source sets

$$\mathcal{S}_{\text{pre}}(x) = D_M, \qquad \mathcal{S}_{\text{ret}}(x) = \mathcal{R}(q).$$

When a contaminant must be explicitly answer-bearing, we additionally require a predicate $g(a, d) \in \{0, 1\}$ (e.g., answer mention or high-overlap), and record positives of

$$\exists\, x, d \ \text{ s.t. } \ f(q, d) \geq \tau_q \ \wedge \ g(a, d) = 1. \tag{2}$$

3.2 Implementation details

We instantiate f using five auditable open-data detectors frequently used in LLM pre-training studies and consolidated in [?]:

- **Exact Match (EM)** [?].
- **8-gram words** (GPT-2 pre-training study) [?].
- **13-gram words** (GPT-3 pre-training study) [?].
- **50-character substring** (GPT-4 technical report) [?].
- **8-gram words with 70% threshold** (PaLM pre-training study) [?].

Span-based detectors scale to long documents, handle the length mismatch between short questions and web pages, and yield concrete evidence spans. While semantic detectors (embeddings / paraphrase) may increase recall, they are harder to audit and become infeasible across disparate lengths; we therefore adopt span-based methods as primary signals, consistent with cautions in [?] and common audit practice [?].

Pretraining channel (candidate retrieval). Full membership tests against all datapoints in web-scale pretraining corpora are infeasible. Following candidate-set approximations in open-data audits [?], we index the Common Crawl C4 corpus in an Elasticsearch-compatible service (AWS OpenSearch). For each $x = (q, a)$, we retrieve top-k candidates with BM25 (default k=1000) and compute contamination using Eq. (??) for *both views* (π_{in} and $\pi_{\text{in+lab}}$), as well as the *answer-bearing* variant Eq. (??) (question view with $g(a, d)$). Any positive flag indicates potential pretraining contamination. This candidate pool serves as $S_{\text{pre}}(x)$.

Retrieval-time channel (web-RAG).. For each q, we query SearxNG [?] with the raw question string, pool results from the first three results pages across underlying engines, and define $\mathcal{R}(q)$ accordingly. We compute contamination over this pool using Eq. (??) for *both views* and Eq. (??) (answer-bearing, question view), and we also use the same pool as retrieval context for answer generation. This isolates run-time leakage introduced by search from memorization in M. Here $S_{\text{ret}}(x) = \mathcal{R}(q)$.

4 DynamicWebQA Framework

We propose a method that leverages *seed graphs* to anchor future question generation while avoiding fixed test items that are easy to memorize or contaminate. At each evaluation round, every graph yields a fresh QA pair produced from the same underlying anchors, so rounds are semantically comparable without reusing static instances.

4.1 Seed Graphs

At evaluation round t, we maintain a set $\mathcal{S}_t = \{G_1^{(t)}, \ldots, G_{n_t}^{(t)}\}$ of seed graphs. Each graph $G_i^{(t)} = (V_i^{(t)}, E_i^{(t)}, \Xi_i^{(t)})$ contains: (i) a node set $V_i^{(t)}$ of *documents* retrieved at evaluation time, (ii) an edge set $E_i^{(t)}$ that records how documents have been jointly used to form past questions, and (iii) metadata $\Xi_i^{(t)}$ (timestamps, operator labels, usage counts). We initialize each graph with document nodes. As new QA items are generated from the graph, we *add edges* between the documents used in that item and label those edges with the applied operator and round. This builds a usage history over time, supports collision control, and keeps nodes aligned with current web sources when refreshed at later rounds.

4.2 Claim Extraction

Given the current document nodes $V_i^{(t)}$, we extract one-sentence, verifiable claims and their supporting spans using a fixed LLM prompt ("Claim Extraction Prompt"). Let

$$C_i^{(t)} = \bigcup_{d \in V_i^{(t)}} \{(d, \text{claim}, \text{span})\}$$

denote the set of extracted claims, bucketed by their source document d. This step yields atomic, referenceable units that we later compose into questions.

4.3 Operator-Controlled QA Composition

Question complexity and the type of reasoning required are jointly controlled by (i) a small set of composition operators O={conjunction, temporal, comparison} and (ii) the hop count h – the number of distinct document buckets used – constrained to $h \in [h_{\min}, h_{\max}]$.

Each operator is realized by a fixed prompt template (see Appendix): conjunction enforces a logical AND so the answer changes if any referenced claim were false; temporal asks for an ordering or interval over time; and comparison requires a contrast (e.g., higher/lower, earlier/later, ratio). We specify $h \in [h_{\min}, h_{\max}]$ to bound multi-hop complexity, with operator-specific constraints enforced by the templates.

4.4 QA Generation

The generation function Gen takes the graph, a random seed, and fixed configuration to produce one QA pair:

$$(q_{i,t}, a_{i,t}, U_{i,t}) = \mathrm{Gen}\big(G_i^{(t)};\ r_{i,t}, \theta\big), \tag{3}$$

where θ fixes the prompt templates and decoding settings (temperature, top-p, stop tokens), and $r_{i,t} \sim \mathcal{R}$ supplies per-sample stochasticity. Internally, Gen: (1) samples an operator $o \in O$ and a hop count h within the configured range; (2) selects h document buckets from $C_i^{(t)}$ and assembles the operator-specific template with the selected claims; and (3) invokes an LLM to return a JSON object containing the list of used claims, the question, and a single concise answer. We write $U_{i,t}$ for the generation trace (operator o, documents used, and claim identifiers). Holding θ fixed and varying only $r_{i,t}$ produces i.i.d. samples per graph while keeping composition policy constant.

4.5 Graph Update and History

After generation, we update the graph to record usage:

$$G_i^{(t+1)} = \mathrm{Update}\big(G_i^{(t)}, U_{i,t}\big),$$

which adds edges between all documents referenced in $U_{i,t}$ and annotates each edge with (o, round t, QA id). This history supports two practical goals: (i) tracking how documents have been combined across rounds and operators, and (ii) reducing collisions by allowing downstream filters to avoid reusing the same document combinations. The node set $V_i^{(t)}$ can also be refreshed to reflect changing sources.

4.6 Sampling and Distributional Properties

In round t, with the configuration θ held fixed (prompts, operator/hop ranges, decoding settings), the only randomness comes from per-graph seeds $r_{i,t} \sim \mathcal{R}$, which drive operator/hop selection, claim selection, and decoding inside Gen. For each graph, this induces a distribution over QA pairs:

$$\mathcal{P}_{i,t} = \mathrm{Dist}\big(\mathrm{Gen}(G_i^{(t)}; r_{i,t}, \theta)\big). \tag{4}$$

With independent seeds, the round-t dataset $D_t = \{(q_{1,t}, a_{1,t}), \ldots, (q_{n_t,t}, a_{n_t,t})\}$ is an independent draw from the product measure

$$\mathcal{P}_t = \bigotimes_{i=1}^{n_t} \mathcal{P}_{i,t}. \tag{5}$$

For a model M and any per-question metric $\mathrm{score}((q,a),M)$ (e.g., EM, F1, log-loss), the aggregate dataset score is

$$\mathrm{Agg}(D_t, M) = \frac{1}{n_t} \sum_{i=1}^{n_t} \mathrm{score}((q_{i,t}, a_{i,t}), M). \tag{6}$$

By linearity of expectation,

$$\mathbb{E}_{D_t \sim \mathcal{P}_t}[\mathrm{Agg}(D_t, M)] = \frac{1}{n_t} \sum_{i=1}^{n_t} \mathbb{E}_{(q,a)\sim\mathcal{P}_{i,t}}[\mathrm{score}((q,a),M)], \tag{7}$$

so the expected aggregate is the average of the per-graph expectations, giving each seed graph equal weight.

4.7 Cross-Round Reporting and Compatibility

Across rounds, the benchmark maintains a persistent set of seed graphs $\mathcal{S}$. As QA items are generated, edges and usage history accumulate. The design remains compatible with evolving sources: nodes represent current documents, so when a source changes, claims can be re-extracted the next time that graph is sampled. Within intervals where the node set and configuration θ are unchanged, successive generations for a given graph are i.i.d.; over longer horizons, the per-graph distribution $\mathcal{P}_{i,t}$ can drift with source evolution. To report results under these conditions, we use two complementary views:

i) Snapshot Evaluation. Choose a reference round t^* and evaluate all systems on its realized dataset D_{t^*}. This fixes both the source set and the random draw, yielding identical test conditions.

ii) Macro-Averaged Score. For longitudinal tracking over a window $W = \{t_1, \ldots, t_T\}$ (e.g., the most recent T rounds), compute

$$\mathrm{Score}_{\mathrm{macro}}(M) = \frac{1}{T} \sum_{j=1}^{T} \mathrm{Agg}\big(D_{t_j}, M\big). \tag{8}$$

Each D_{t_j} is an i.i.d. draw from its round-specific $\mathcal{P}_{t_j}$, so this macro-average summarizes average effectiveness under natural benchmark evolution without letting any single round dominate.

4.8 Verifying Distributional Consistency

Under a stable generation policy (with θ held fixed), the embedding geometry should be invariant across rounds. Sentence embeddings are used as a practical proxy for topical relatedness, where proximity often reflects overlapping content or cues. Consequently, if the topic mix is stable, the distribution of pairwise distances should remain unchanged across rounds. Topic composition can change in ways that leave the average embedding unchanged—for example, similar topics in different proportions or broader within-topic spread—yet these changes alter the pattern of pairwise distances. Round-wise differences are therefore assessed using a distance-based permutation MANOVA (PERMANOVA) [**?**] as the omnibus test, and pairwise differences using the energy two-sample test [**?**].

Embedding and distances. Each QA pair is concatenated and embedded with a sentence embedding model (`intfloat/e5-base-v2` [**?**] in our experiments). Embeddings are ℓ_2-normalized, so Euclidean and cosine distances are monotone-equivalent. Let D denote the Euclidean distance matrix over all items.

Omnibus test across K rounds. PERMANOVA is applied to D with round labels (10,000 permutations). Reported statistics are the pseudo-F, permutation p-value, and R^2. This serves as a K-sample omnibus test for differences in location and dispersion in the distance space.

Pairwise localization. For all $\binom{K}{2}$ round pairs, the energy two-sample test is applied on D (10,000 permutations) with Benjamini–Hochberg FDR control at α=0.05. Reported quantities are the E-statistic and permutation p-value.

Descriptive visuals. (i) A 2D PCA of embeddings colored by round (descriptive intermixing). (ii) An energy–distance heat map summarizing cross-round separation after accounting for within-round dispersion; larger off-diagonal values indicate greater separation.

4.9 LLM-as-a-Judge for Annotations

Automated annotation is performed with three independent LLMs acting as judges using a fixed *LLM-as-Judge* prompt (Appendix). For each QA instance, the prompt elicits a brief rationale and returns four binary flags with the convention that 1 denotes a desirable property: Logical Structure (well-formed and unambiguous question), Non-Redundancy (the question does not contain or tautologically imply its answer), Evidence Support (the answer is supported by the cited claims and source spans from the graph's document nodes), and Answer Adequacy (the answer fully addresses the question and is of the correct type). Rationales are collected for auditing and are motivated by prior findings that explanations can improve judgment quality, but only the binary flags are used for filtering [? ?].

Aggregation requires unanimous agreement: all three judges must set all four flags to 1. In addition, the item must pass schema validation, citation traceability (document and claim identifiers with spans), and operator-specific checks such as hop-count bounds. Items failing any condition are discarded, and generation is retried for that graph until one item passes. This procedure ensures that each graph contributes exactly one accepted QA pair per round, preserving equal per-graph weighting in reporting and maintaining the distributional assumptions stated earlier.

5 Results

5.1 Contamination Modeling in QA Datasets

5.2 Model Performance

5.3 Distributional Consistency: Results

Omnibus. PERMANOVA detects no round effect in either split (Table ??). With 10,000 permutations the p-value resolution is 10^{-4}; both splits yield $p \geq 0.99$ and $R^2 \leq 0.134\%$, indicating negligible variance explained by "round".

Pairwise. For the 45 round pairs per split, no comparison is significant after BH–FDR (α=0.05). E-statistics are tiny (10^{-4}–10^{-3}) and centered near zero (Table ??); maxima are $< 10^{-3}$.

Visual evidence. PCA projections (Fig. ??) show full intermixing by round (PC1/PC2 <4% explained each). Drift heatmaps of signed E-statistics (Fig. ??) concentrate near zero without structure. Centered mean-distance maps and relative deviations (Fig. ??) show max absolute effects $\leq 1.5\times10^{-3}$, i.e., <0.25% of the within-round scale.

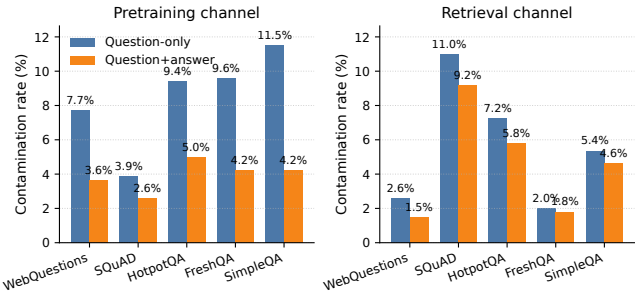

Figure 1: Contamination rates by dataset across the two leakage channels defined in the modeling framework: (a) pretraining contamination (source set $\mathcal{S}_{\text{pre}}$=D_M), and (b) retrieval channel contamination (source set $\mathcal{S}_{\text{ret}}$=$\mathcal{R}(q)$).

	Dynamic (R1–R10 mean)			Paraphrased
Model	Base	Web-RAG		
Delta	Base	Web-RAG		
Delta	Base	Web-RAG	Base	Web-RAG
openai.gpt-oss-20b-1:0	0.277	**0.494**	**+0.217**	0.339
openai.gpt-oss-120b-1:0	0.396	**0.526**	**+0.130**	0.442
mistral.mistral-7b-instruct-v0:2	0.171	**0.278**	**+0.107**	0.301
mistral.mistral-large-2402-v1:0	0.317	**0.429**	**+0.112**	0.431
us.meta.llama4-scout-17b-instruct-v1:0	0.277	**0.485**	**+0.208**	0.388
us.meta.llama4-maverick-17b-instruct-v1:0	0.362	**0.523**	**+0.161**	0.470

Table 1: Means over rounds 1–10 (Dynamic, Paraphrased) and single-shot Original Splits (High contamination, Random). *Delta* is Web-RAG minus Base. Ordering follows plots: GPT (small *rightarrow* large), Mistral (small *rightarrow* large), Llama (small *rightarrow* large).

Table 2: Omnibus PERMANOVA across K=10 rounds (Euclidean on ℓ_2-normalized embeddings; 10k permutations).

Split	N	Pseudo-F	R^2 (%)	Perms	p-value
Highly Contaminated	5,000	0.7429	0.1338	10,000	≥ 0.99
Randomly Sampled	4,996	0.6476	0.1168	10,000	≥ 0.99

Table 3: Summary of pairwise Energy tests (signed, U-centered E-statistic).

Split	#Sig (FDR)	Mean	Median	Std
Highly Contaminated	0/45	-5.14×10^{-4}	-6.31×10^{-4}	3.16×10^{-4}
Randomly Sampled	0/45	-6.58×10^{-4}	-7.10×10^{-4}	2.08×10^{-4}

Model performance summary. Retrieval consistently improves accuracy across all models and settings. Gains on Dynamic range from +0.107 to +0.217 and on Paraphrased from +0.172 to +0.287.

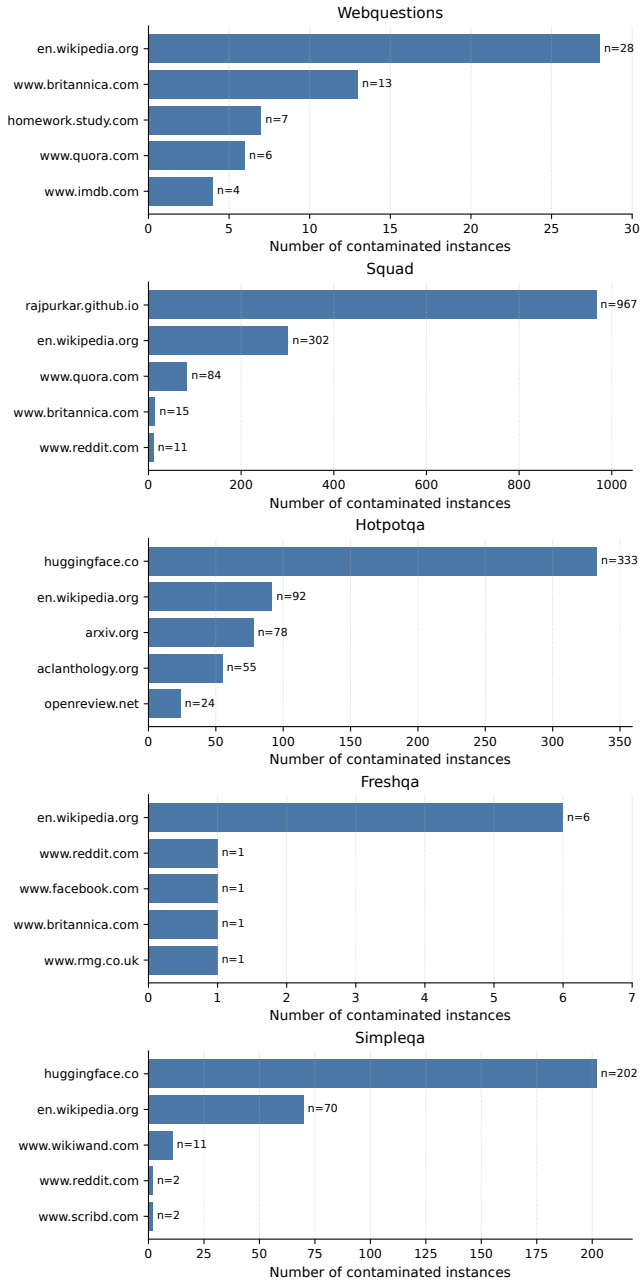

Figure 2: Top contamination sources during retrieval for each dataset. For each dataset, we show the top-5 domains by number of contaminated instances.

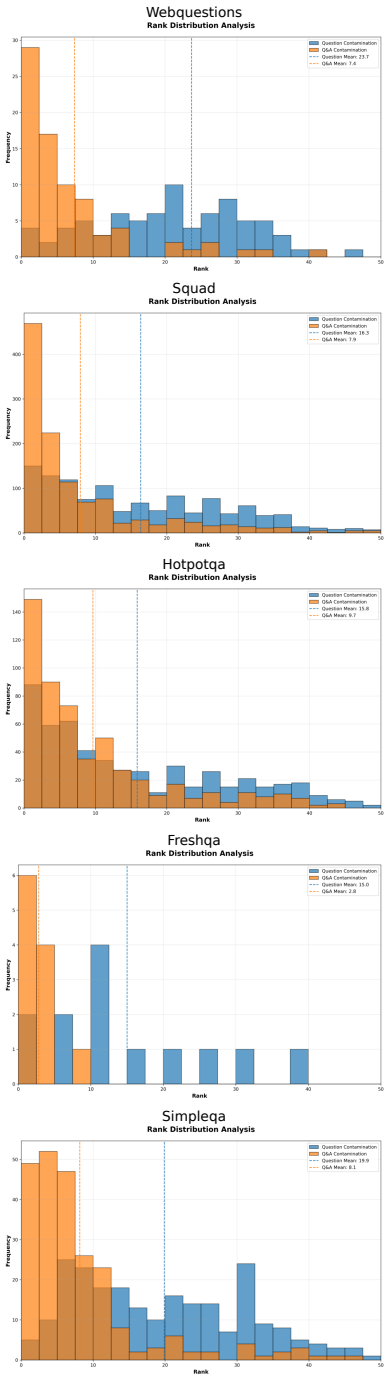

Figure 3: Rank distribution of contaminated evidence surfaced by the retriever, shown per dataset. Histograms separately show question-only and question+answer contamination where available.

The largest lifts occur for mid-size models (e.g., +0.217 for GPT-20B Dynamic; +0.208 for Llama4-Scout-17B Dynamic), while larger models still benefit but less. Paraphrased is uniformly easier than Dynamic for a fixed setting, indicating sensitivity to the distribution shifts introduced by dynamic variants. Original Split scores are substantially higher than evolving rounds; High-contamination is

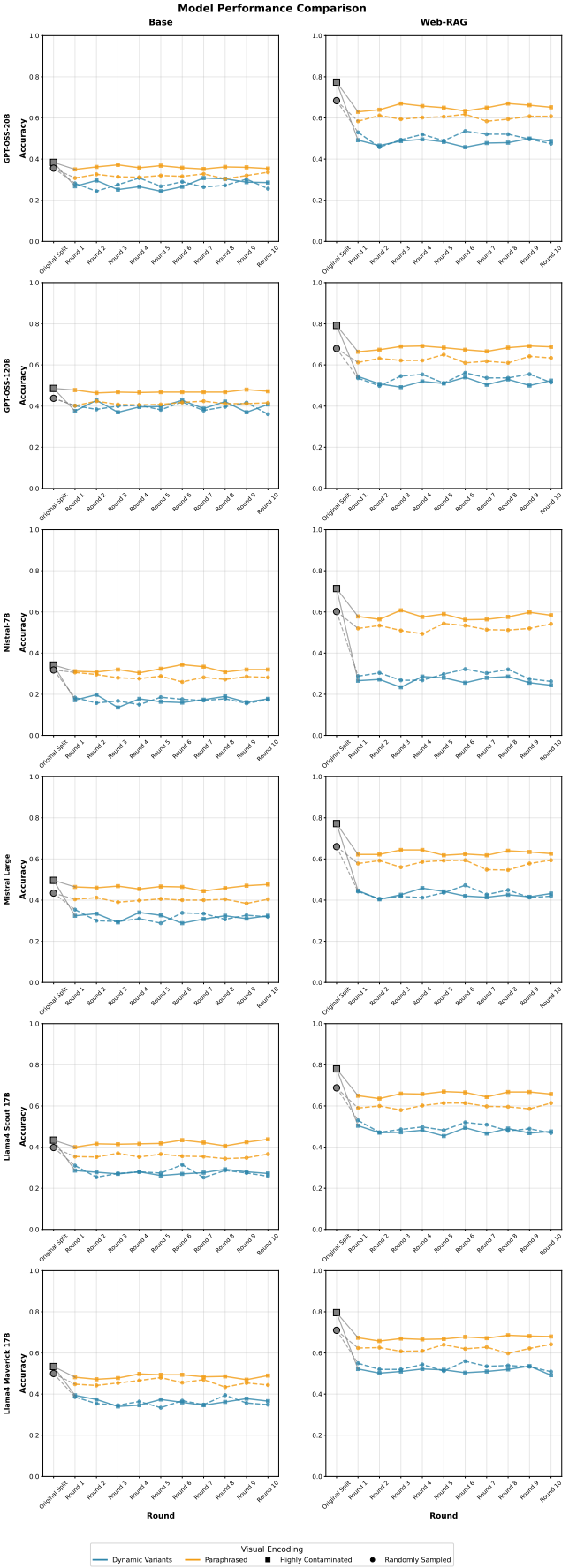

Figure 4: Caption TBD

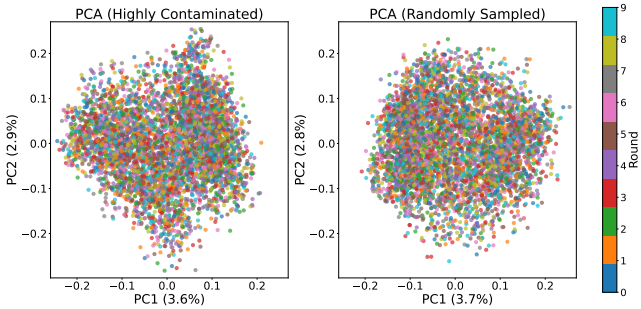

Figure 5: PCA projections of ℓ_2-normalized embeddings for Highly Contaminated (left) and Randomly Sampled (right) splits. Points are colored by round; both splits show full intermixing with PC1/PC2 each explaining <4% variance.

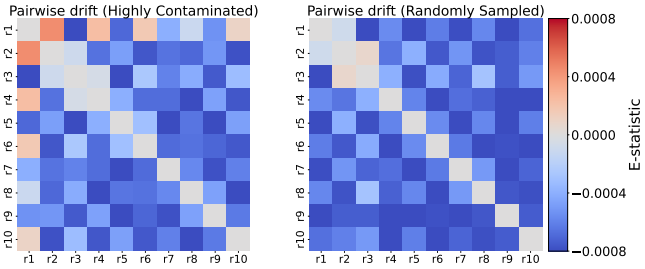

Figure 6: Pairwise drift heatmaps. Left: Highly Contaminated; Right: Randomly Sampled. Cells show the *E-statistic* from the Energy two-sample test (`hyppo.ksample.Energy`) computed on Euclidean distances of ℓ_2-normalized embeddings; 10k permutations with BH–FDR over 45 round pairs.

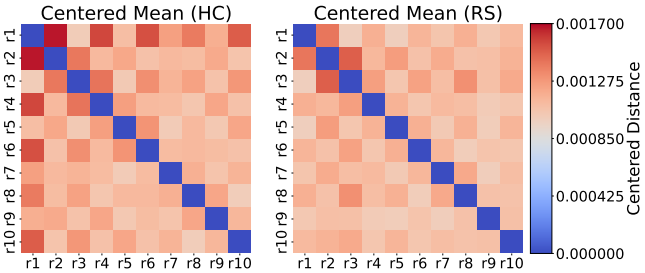

Figure 7: Centered mean distance across rounds. Left: Highly Contaminated; Right: Randomly Sampled. Each cell shows cross-round mean distance minus the average within-round distance (proportional to the biased Energy distance). Values cluster near zero with maxima $\leq 1.7\times10^{-3}$.

highest, Random is lower but still above Dynamic/Paraphrased, underscoring the challenge posed by our dynamic evaluation.

6 Conclusion

TBD

Appendix

Claim Extraction Prompt

You are given a passage of text. Your task is to extract one-sentence standalone factual claims from the passage.
Each claim must:

- Be self-contained (i.e., understandable on its own without requiring coreference resolution)
- Not rely on ambiguous references like "this", "that", "they", etc.
- Be phrased as a complete declarative sentence
- Be verifiable (e.g., "Riot Games has no plans for League of Legends 2." is acceptable; "They have no plans for League of Legends 2" is not)

For each claim, also provide the exact verbatim supporting text span from the original text that led you to extract the claim.
Output JSON Keys:

- `"claim1"`: standalone claim
- `"supporting_text_span1"`: verbatim supporting text span from original
- `"claim2"`: standalone claim
- `"supporting_text_span2"`: verbatim supporting text span from original
- `...`

Task:
Here is the input text: `{text}`

Temporal-Sequence Multi-Hop QA Prompt

You are generating `{n_pairs}` **temporal-sequence** multi-hop QA pairs.
Buckets of claims (grouped by doc_id) are below:
`{json.dumps(buckets, indent=2)}`

Rules for each QA pair:

- Combine claims from at least **{min_buckets} different buckets**.
- Each chosen claim must mention a year, date, or explicit timeframe.
- Ask for an ordering or an interval (e.g., "How many years after …?").
- Every referenced fact must be essential for the answer.
- The question should not contain the answer or steps to get the answer.
- The question should not refer to the documents, they should be general.
- List every claim you used by its doc_id and claim_id.
- Provide one concise answer (year, number of years, earlier event, etc.).

Output format (JSON list):

- `"used_claims"`: list of {{doc_id, claim_id, claim}}
- `"question"`: string
- `"answer"`: string

Return only the JSON—no extra commentary.

Comparison-Style Multi-Hop QA Prompt

You are generating `{n_pairs}` **comparison-style** multi-hop QA pairs.
Buckets of claims (grouped by doc_id) are below:
`{json.dumps(buckets, indent=2)}`

Rules for each QA pair:

- Use claims from at least **{min_buckets} different buckets**.
- Frame the question so the solver must contrast values (higher, lower, difference, ratio, earlier, later).
- Keep wording tight—only include values essential for the comparison.
- The question should not contain the answer or steps to get the answer.
- The question should not refer to the documents, they should be general.
- List every claim you used by its doc_id and claim_id.
- Provide a single, concise answer (number, entity, etc.).

Output format (JSON list):

- `"used_claims"`: list of {{doc_id, claim_id, claim}}
- `"question"`: string
- `"answer"`: string

Return only the JSON—no extra commentary.

Conjunction-Style Multi-Hop QA Prompt

You are generating `{n_pairs}` **conjunction-style** multi-hop QA pairs.
Buckets of claims (grouped by doc_id) are below:
`{json.dumps(buckets, indent=2)}`

Rules for each QA pair:

- Combine facts from at least **{min_buckets} different buckets** (can use more).
- The answer must change if any referenced claim were false (logical AND).
- No trivia—every fact must be essential.
- The question should not contain the answer or steps to get the answer.
- The question should not refer to the documents, they should be general.
- List every claim you used by its doc_id and claim_id.
- Provide a single, concise answer (entity, number, date, etc.).

Output format (JSON list):

- `"used_claims"`: list of {{doc_id, claim_id, claim}}
- `"question"`: string
- `"answer"`: string

Return only the JSON—no extra commentary.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009